FINAL INTERNSHIP PROJECT REPORT

Secure Inter-VPC Connectivity Using AWS VPC Peering

A two-VPC network with a public bastion host, private EC2 instances, controlled SSH access, and bidirectional ICMP validation

Prepared by: Mayuri Chavan
Program: B.Tech — Computer Engineering
Cloud Platform: Amazon Web Services (AWS)
Region: Asia Pacific (Mumbai), ap-south-1
Project dates: 25–26 October 2026
Submission details: [Institution / internship batch / mentor — fill in]

Executive Summary

This internship lab implemented two isolated Amazon Virtual Private Clouds and connected them using VPC peering. VPC A hosted a public bastion and a private application instance; VPC B hosted a second private application instance. The environment used subnet-specific route tables, an internet gateway for bastion access, and narrowly scoped security-group rules. After the peering connection was accepted and reciprocal routes were added, SSH access through the bastion and successful ICMP traffic in both directions were demonstrated in the supplied screenshots. The three EC2 instances were subsequently stopped.

1. Problem Statement and Objectives

Private resources in separate VPCs cannot communicate merely because their address ranges are known. The exercise required a secure inter-VPC path without assigning public IPv4 addresses to the application servers.

- Create two non-overlapping VPCs and three subnets.
- Provide administrative access through a public bastion while keeping both application servers private.
- Connect the VPCs using a same-region VPC peering connection and configure reciprocal routes.
- Validate access using SSH and demonstrate ICMP reachability in both directions.
- Preserve evidence and stop instances to reduce compute costs.

2. Architecture and Addressing

Logical topology (not to scale):

- Administrator laptop → SSH TCP/22 → Bastion (10.10.1.10, public subnet, VPC A)
- Bastion → SSH TCP/22 → App-A (10.10.2.10, private subnet, VPC A)
- Bastion → VPC peering → SSH TCP/22 → App-B (10.20.1.10, private subnet, VPC B)
- App-A ↔️ VPC peering ↔️ App-B (ICMP Echo Request / Reply)

Component	Name	CIDR / private IPv4
VPC A	ewb-vpc-a	10.10.0.0/16
VPC B	ewb-vpc-b	10.20.0.0/16
Public subnet A	ewb-a-public	10.10.1.0/24
Private subnet A	ewb-a-private	10.10.2.0/24
Private subnet B	ewb-b-private	10.20.1.0/24
Bastion EC2	ewb-bastion	10.10.1.10
Application EC2 A	ewb-app-a	10.10.2.10
Application EC2 B	ewb-app-b	10.20.1.10

VPC identifiers observed during implementation: VPC A vpc-0b850fd94e6a85949; VPC B vpc-04f866a91644c5041. Instances used Amazon Linux 2023 and t3.micro. The original SSH key-pair name was ewb-keypair; private key material is intentionally excluded from this report.

3. VPC and Subnet Creation

The two VPCs use non-overlapping IPv4 address ranges. VPC A contains both a public and a private subnet, while VPC B contains a private subnet.

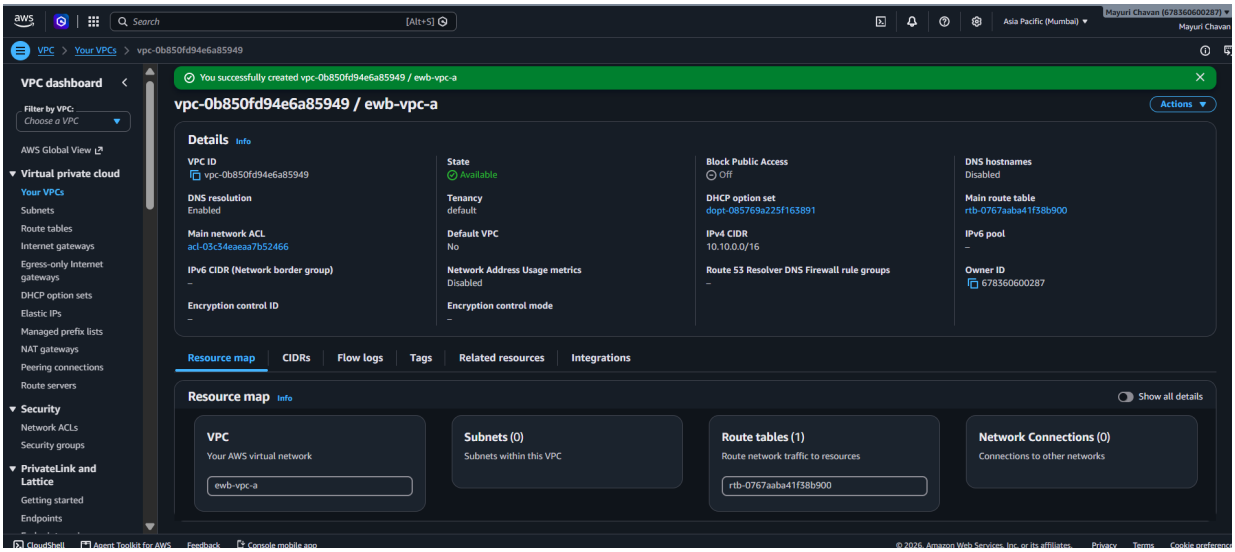


Figure 1. Creation of VPC A.

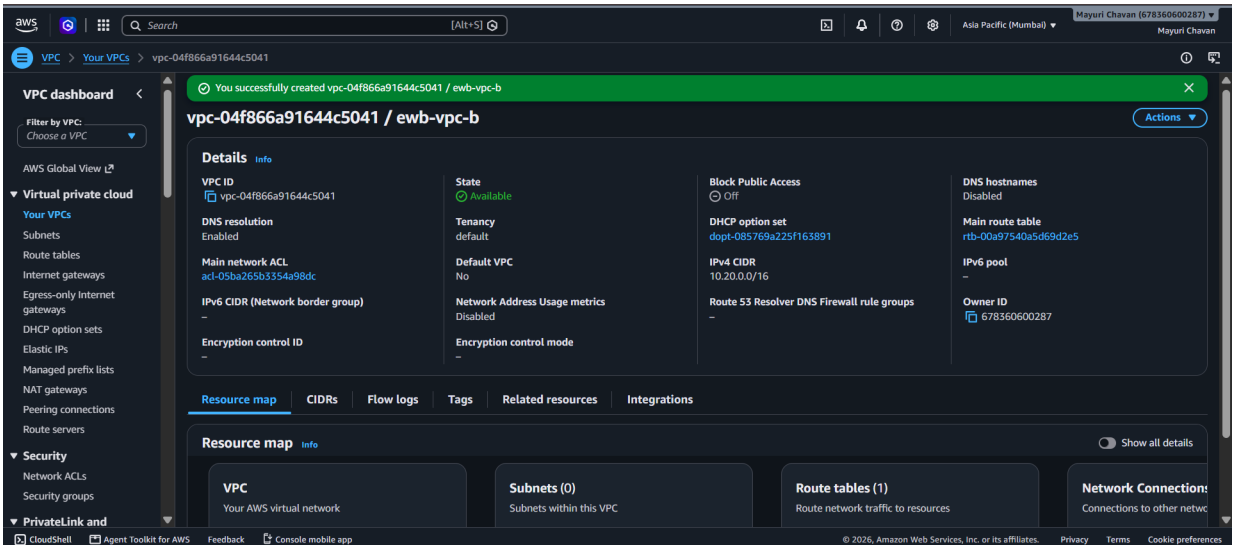


Figure 2. Creation of VPC B.

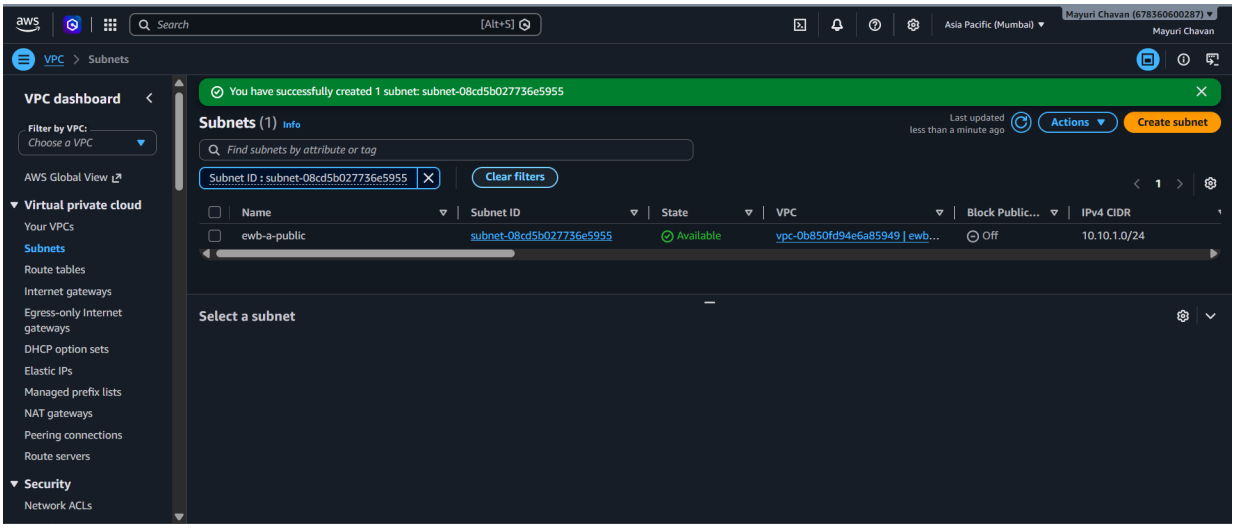


Figure 3. Public subnet in VPC A.

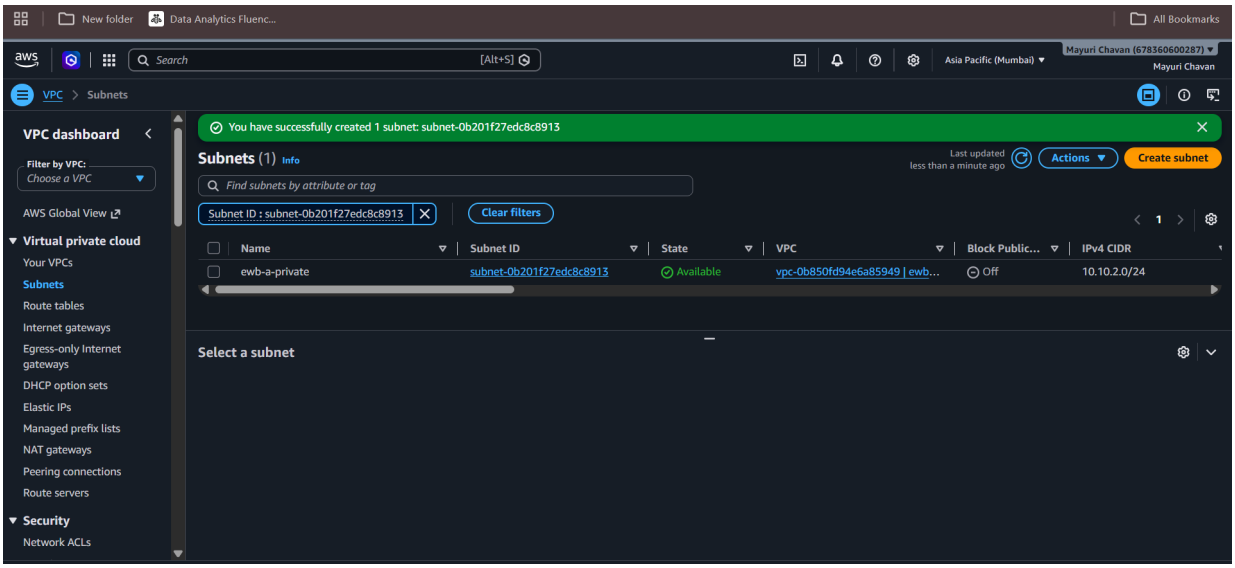


Figure 4. Private subnet in VPC A.

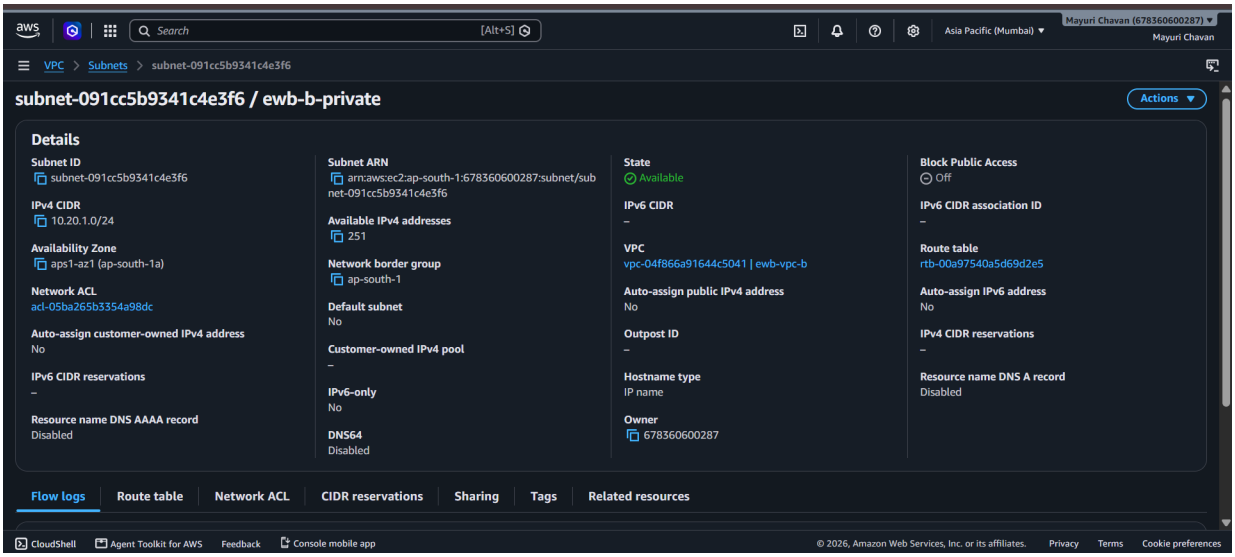


Figure 5. Verified private subnet in VPC B.

4. Internet Gateway and Route Tables

The public route table includes a default route to the VPC A internet gateway; private subnets do not require direct internet access for this test.

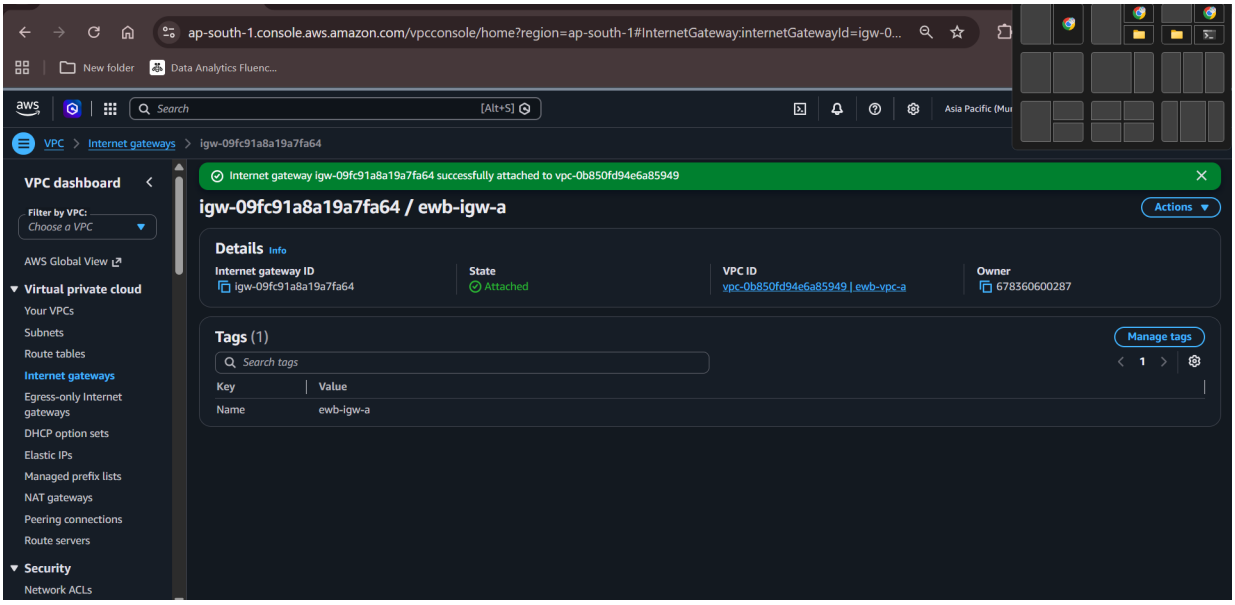


Figure 6. Internet gateway attached to VPC A.

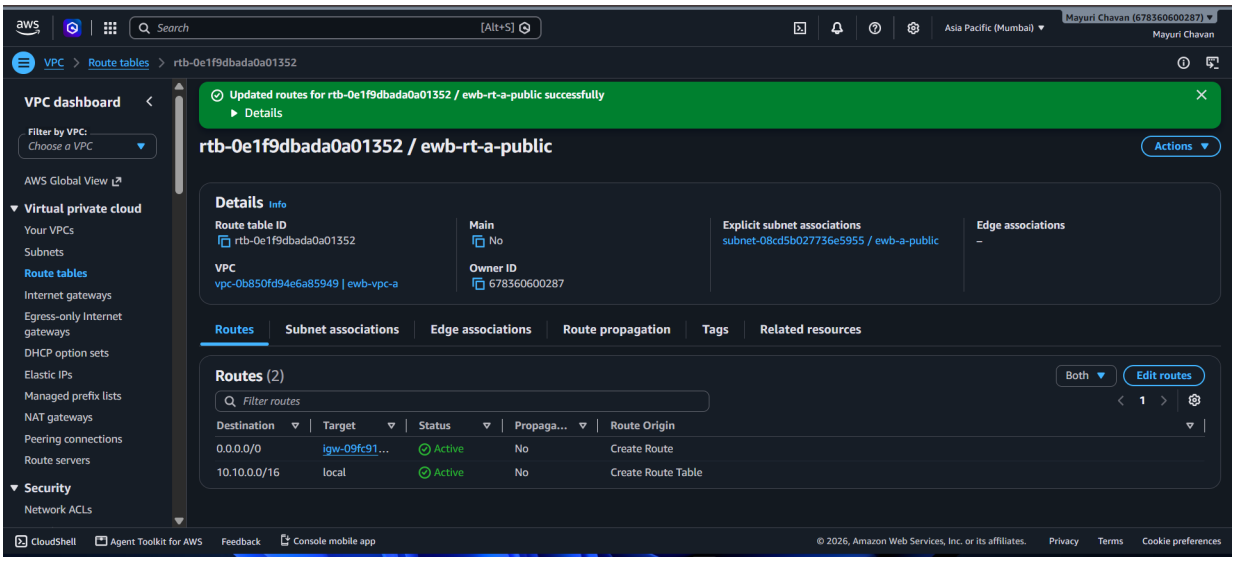


Figure 7. Public route table in VPC A.

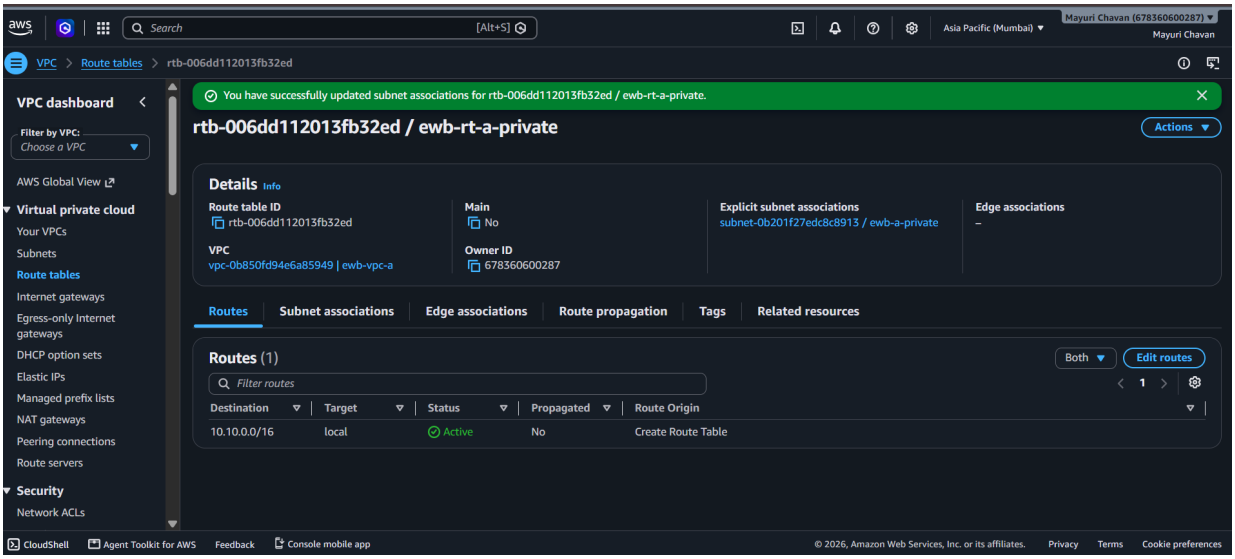


Figure 8. Private route table in VPC A.

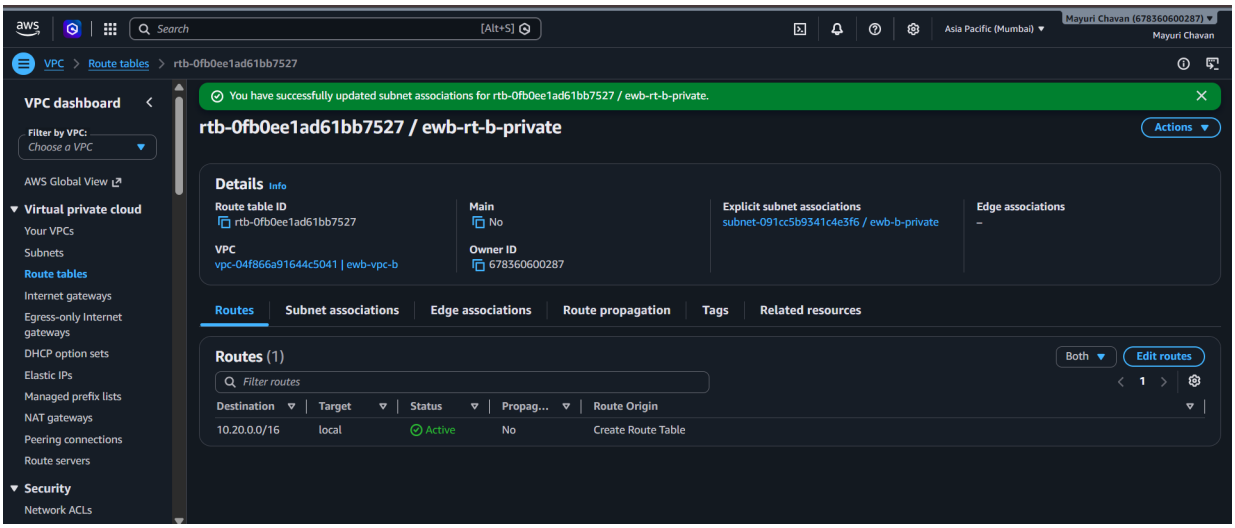


Figure 9. Private route table in VPC B.

5. Security Controls

Security group	Relevant inbound permissions
ewb-sg-bastion	SSH TCP/22 from administrator public IPv4 /32 (as configured)
ewb-sg-app-a	SSH TCP/22 from 10.10.1.10/32; ICMP Echo Request from 10.20.1.10/32
ewb-sg-app-b	SSH TCP/22 from 10.10.1.10/32; ICMP Echo Request from 10.10.2.10/32

AWS security groups are stateful, so responses to permitted connections can return without separate inbound response rules. The application instances were launched without public IPv4 addresses. No private key is reproduced in the report.

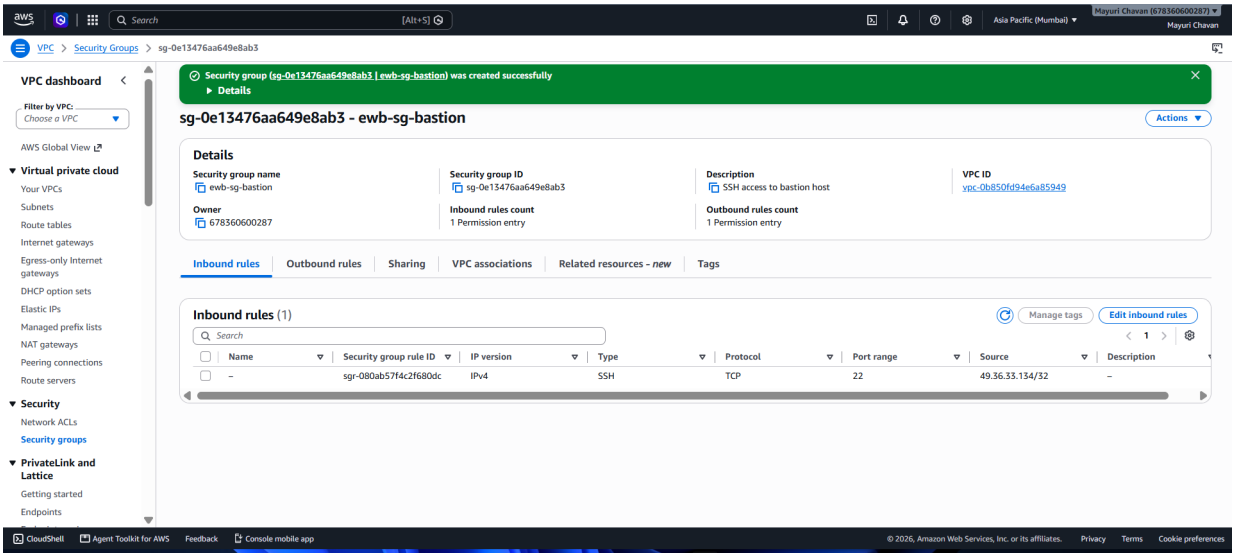


Figure 10. Bastion security group configuration.

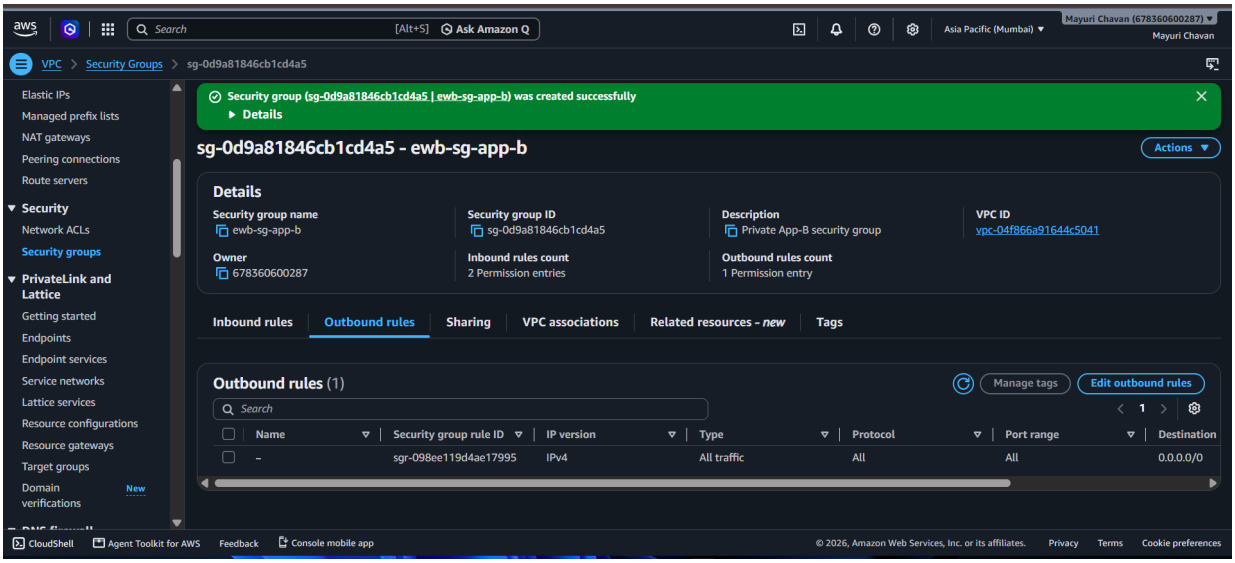


Figure 11. App-B security group configuration.

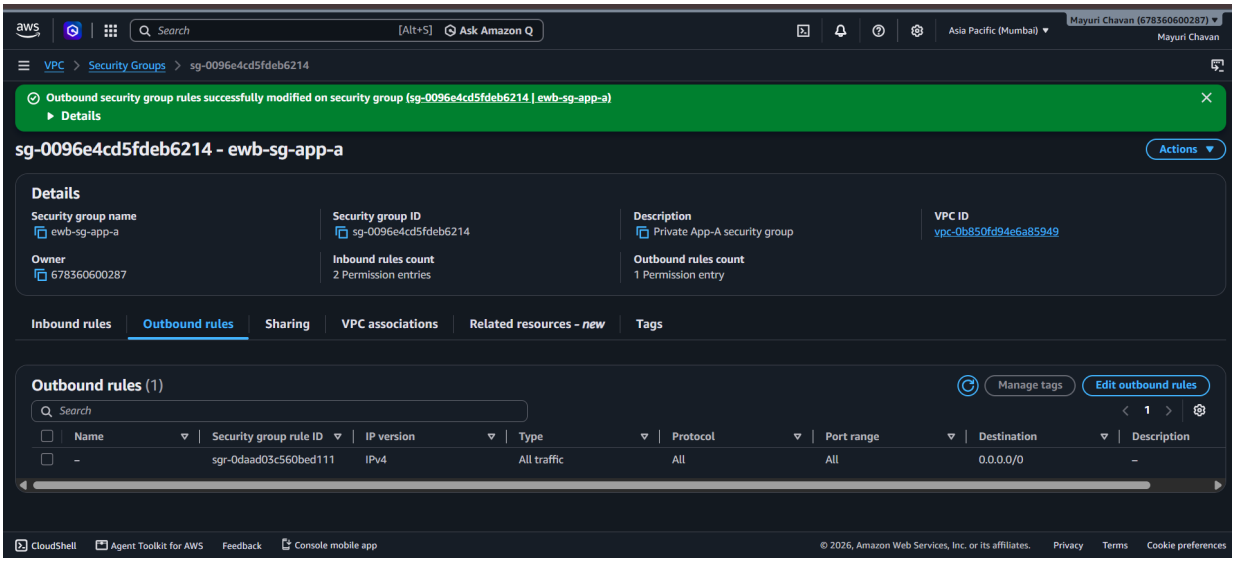


Figure 12. App-A outbound security group configuration.

6. EC2 Provisioning

Three Amazon Linux EC2 instances were provisioned: one public bastion and two private application instances. The key-pair download was initially difficult to locate because it had been saved under a project Screenshots/.ssh directory; it was subsequently located and secured.

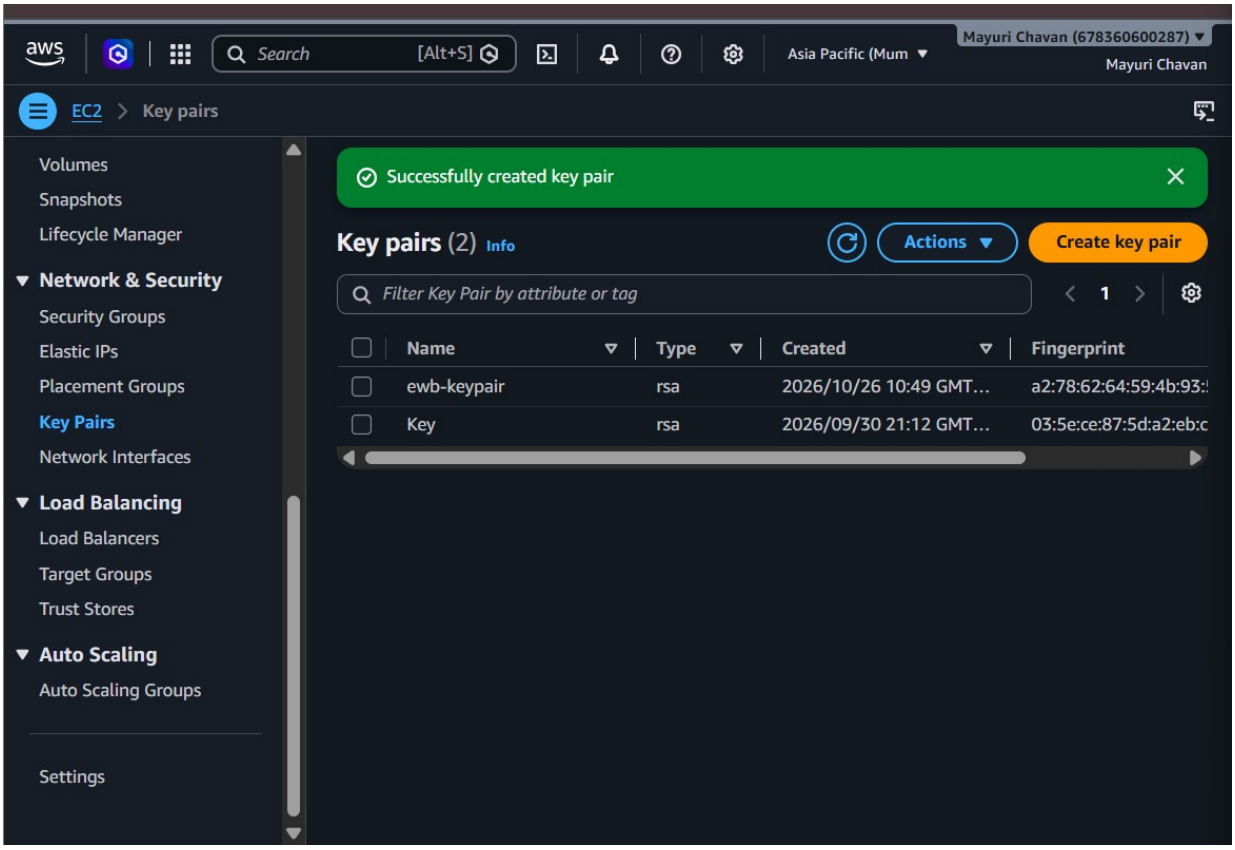


Figure 13. AWS key pair registered (no private key material shown).

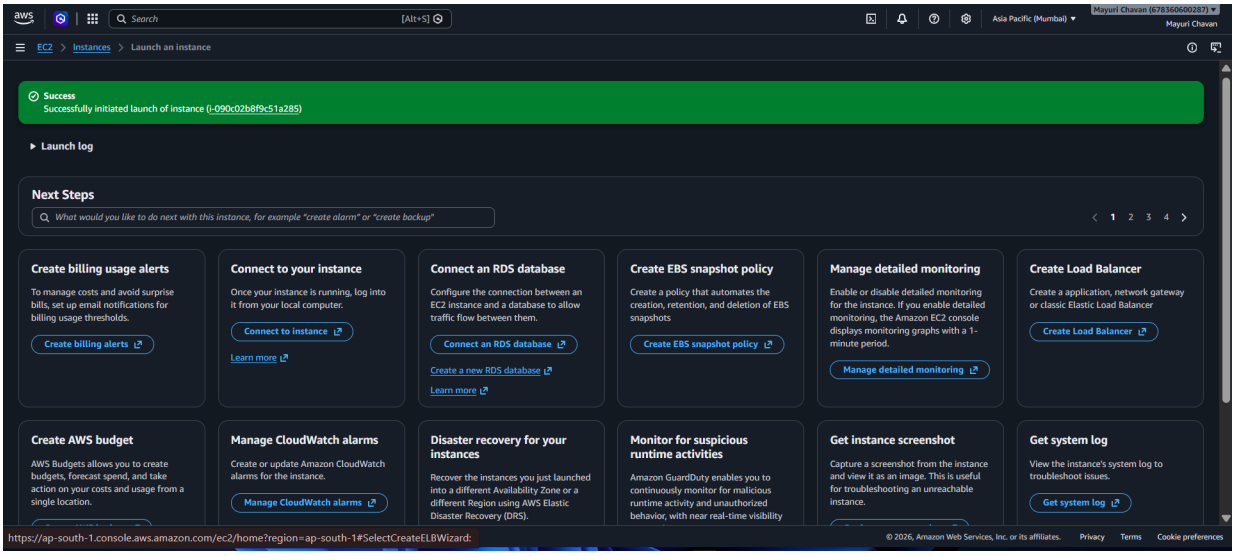


Figure 14. Bastion instance launch confirmation.

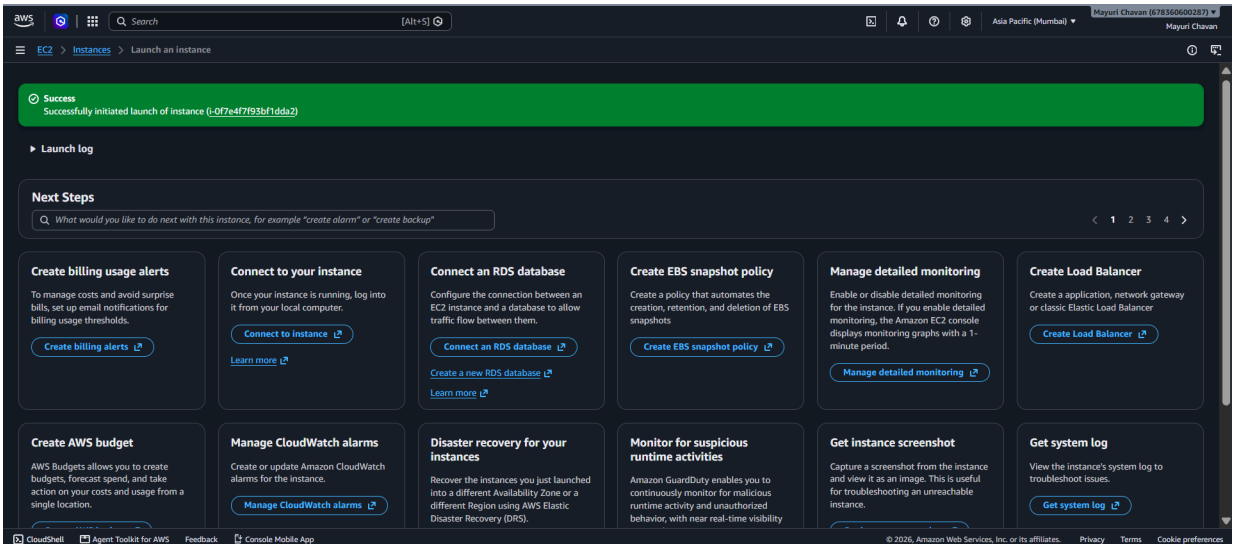


Figure 15. App-A instance launch confirmation.

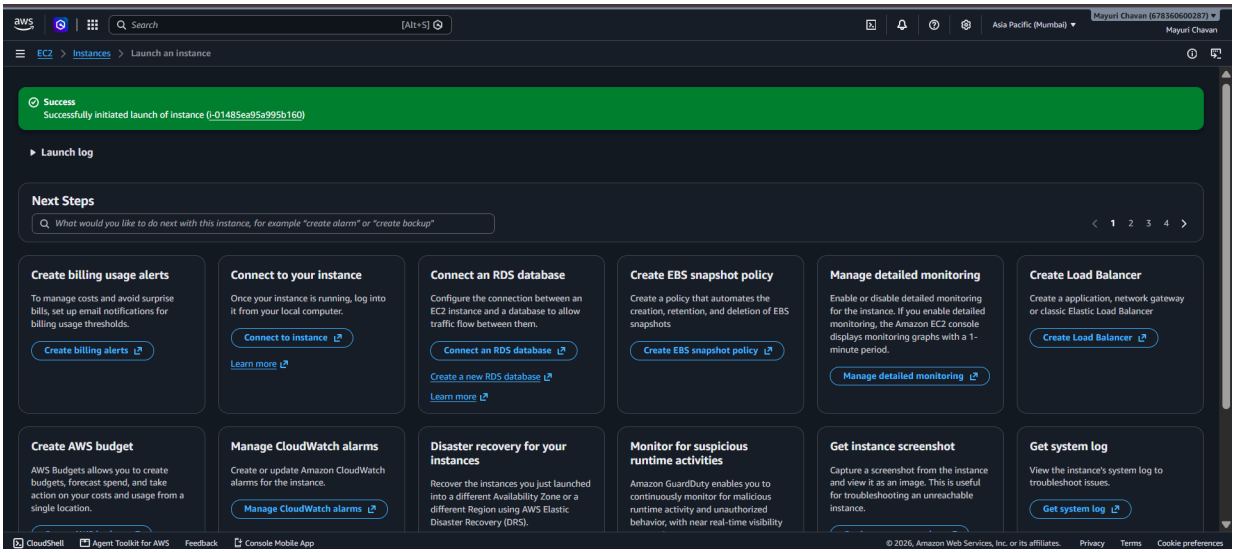


Figure 16. App-B instance launch confirmation.

7. VPC Peering and Routing

A same-region VPC peering connection was created between ewb-vpc-a and ewb-vpc-b, accepted, and shown as Active. The route tables were updated as follows:

Route table	Peering destination	Target
ewb-rt-a-private	10.20.0.0/16	VPC peering connection
ewb-rt-b-private	10.10.0.0/16	VPC peering connection
ewb-rt-a-public	10.20.0.0/16	VPC peering connection

The public table retains 0.0.0.0/0 → internet gateway, while local VPC routes remain in place. These peering routes permit the bastion to reach App-B and allow application traffic between VPCs.

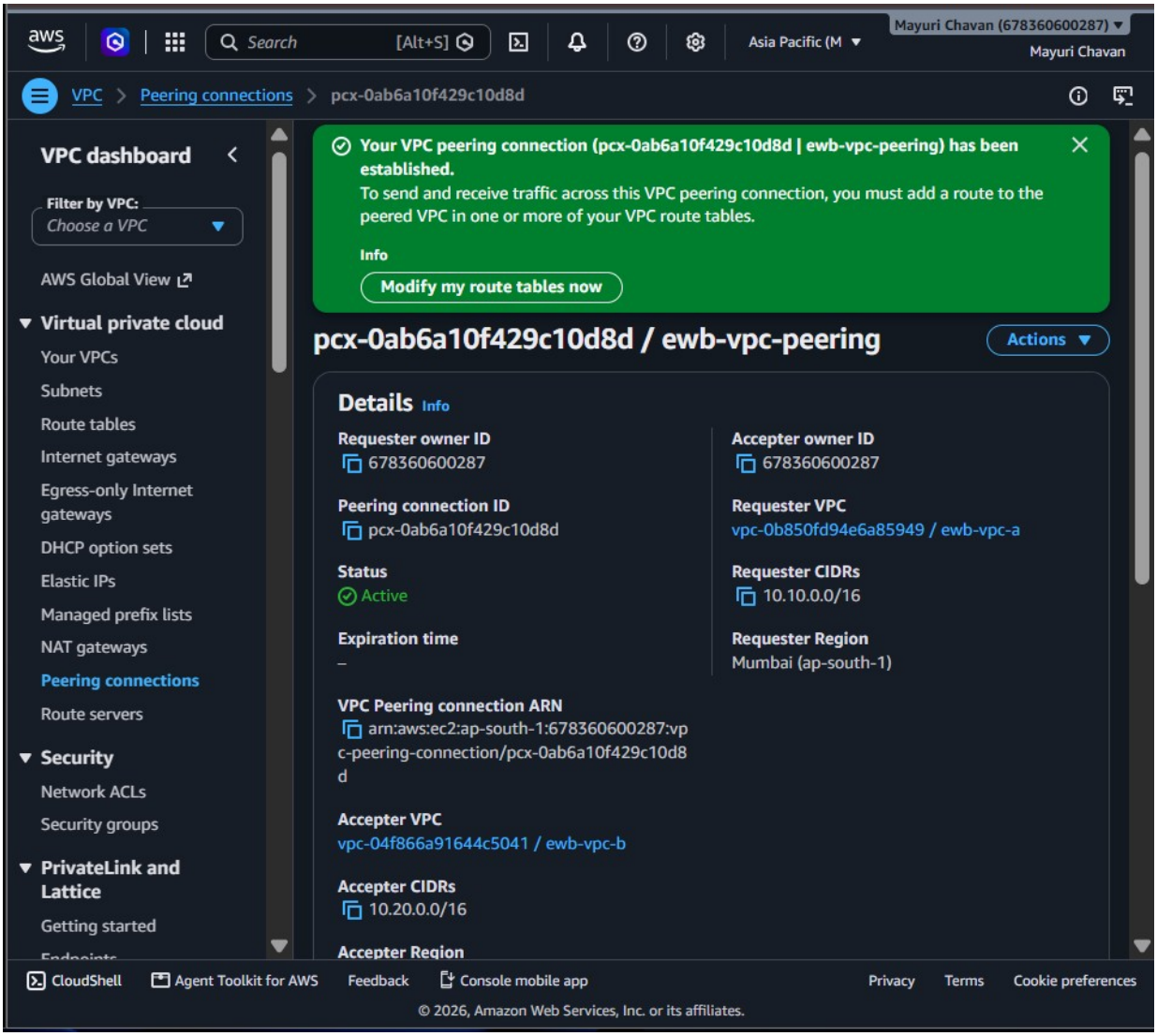


Figure 17. Accepted and active VPC peering connection.

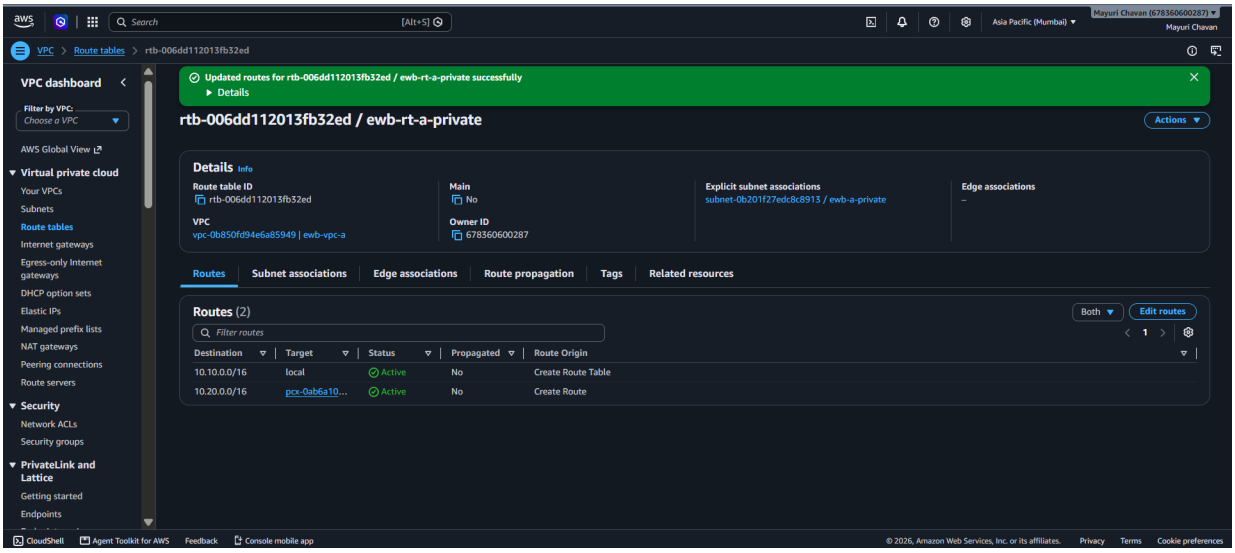


Figure 18. Peering route added to private route table A.

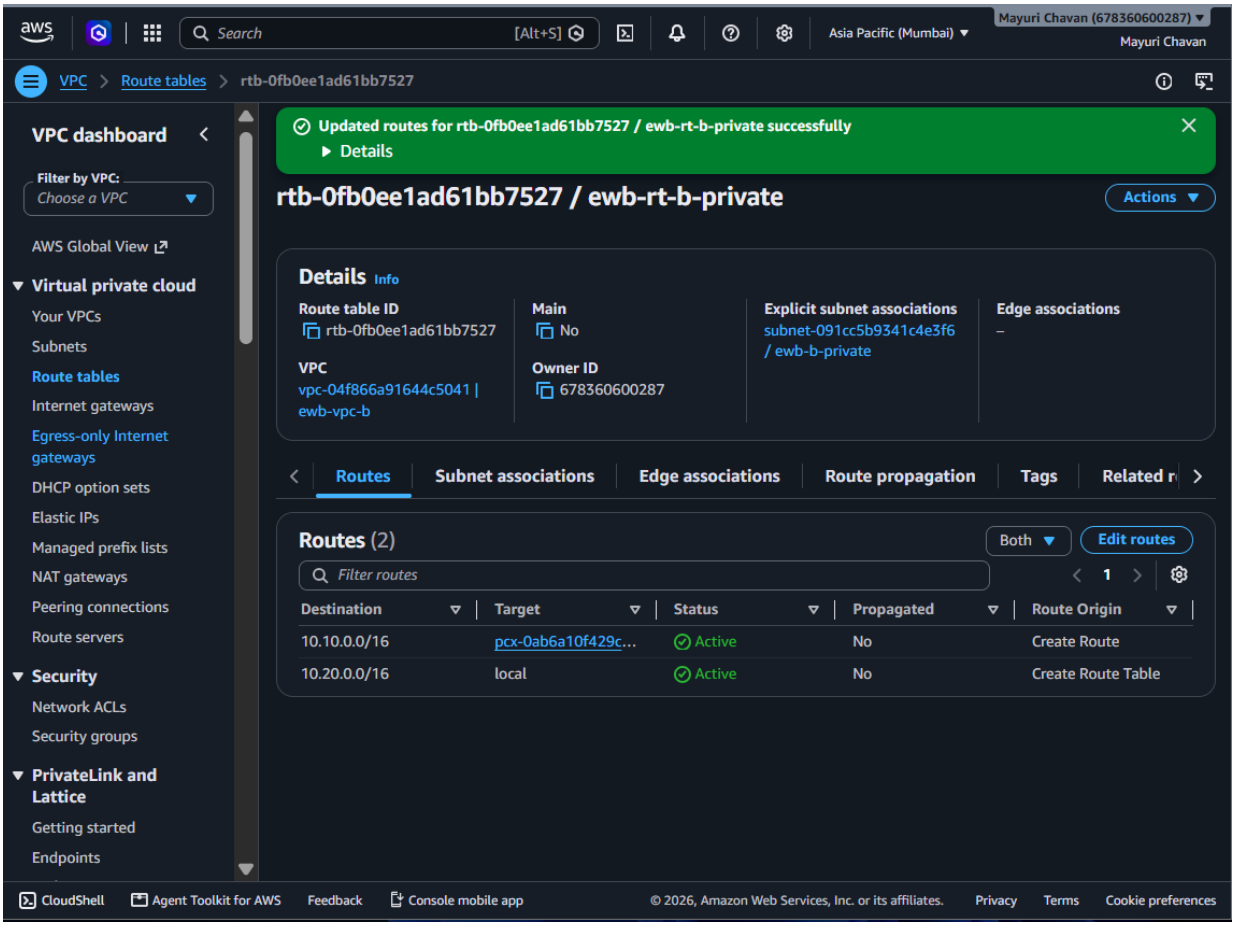


Figure 19. Peering route added to private route table B.

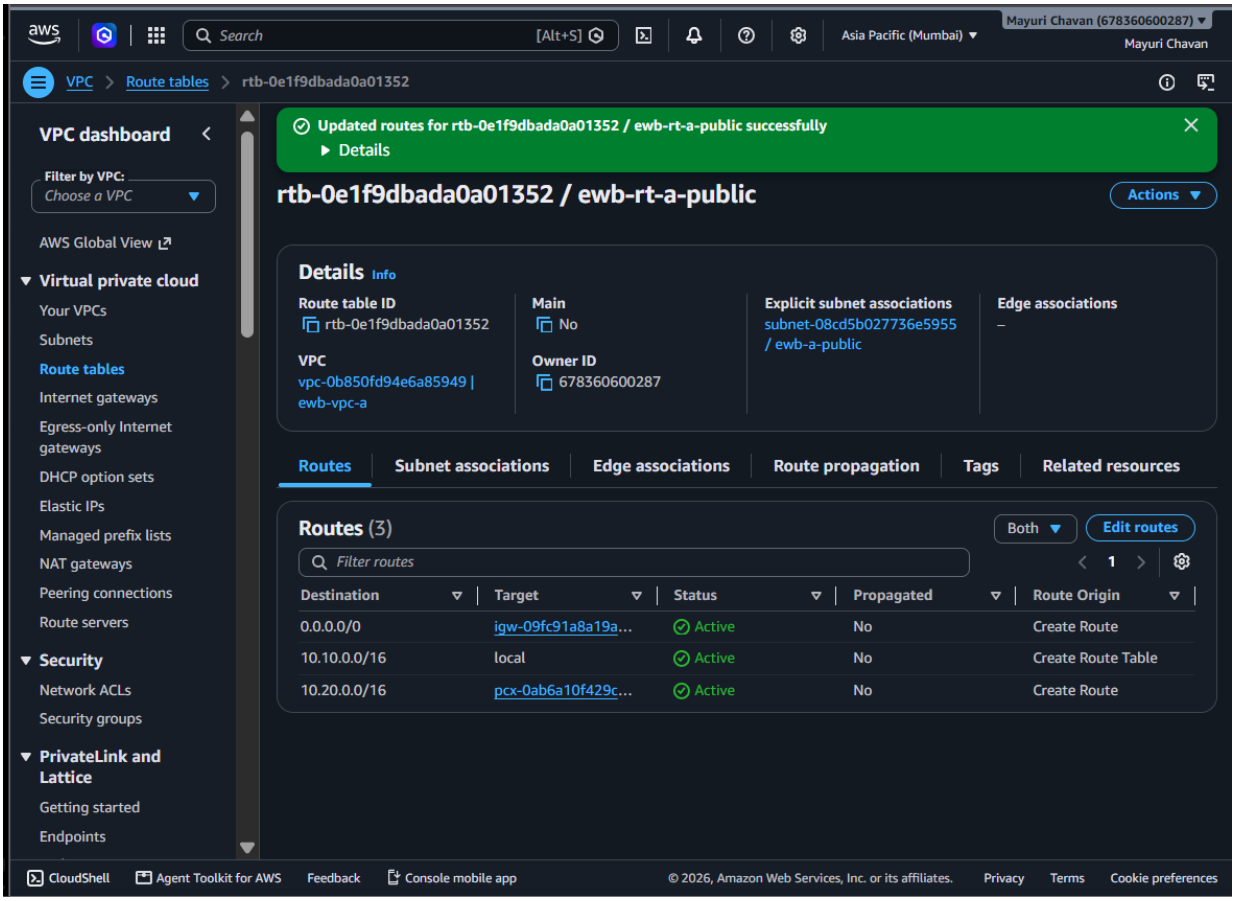


Figure 20. Peering route added to bastion public route table.

8. SSH Access and Connectivity Validation

SSH connectivity was established from Windows PowerShell to the bastion and then through the bastion to each private EC2 instance. The uploaded terminal captures demonstrate successful login to App-A and App-B. The application instances then exchanged ICMP Echo Requests over the peering connection.

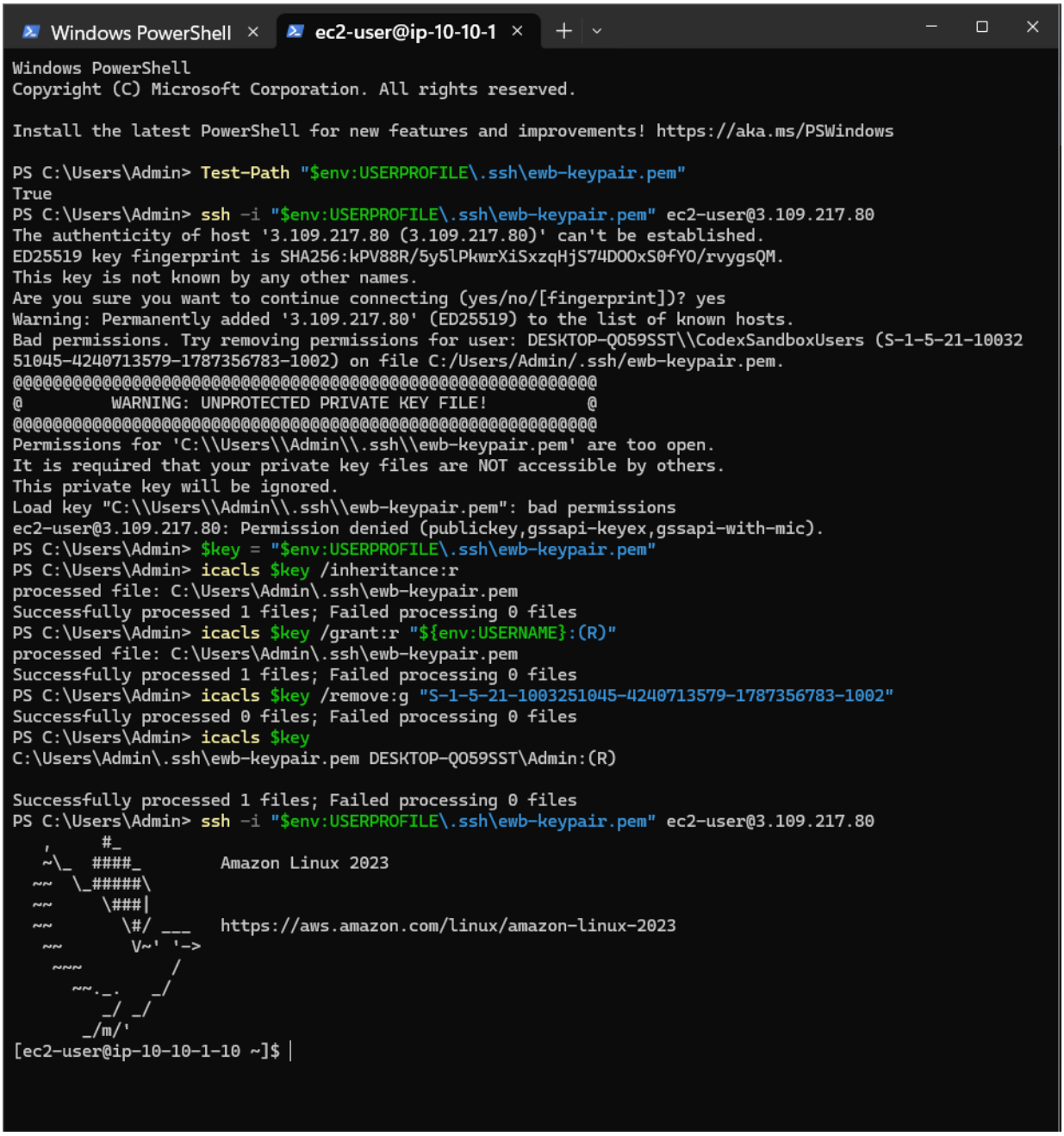


Figure 21. Successful bastion SSH session.

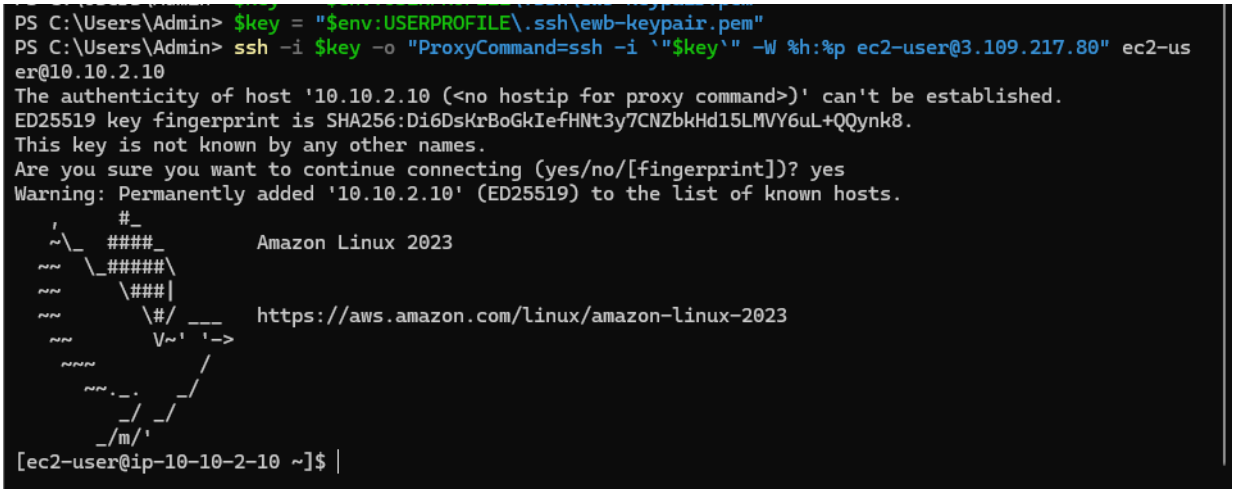


Figure 22. Successful SSH session on App-A through the bastion.

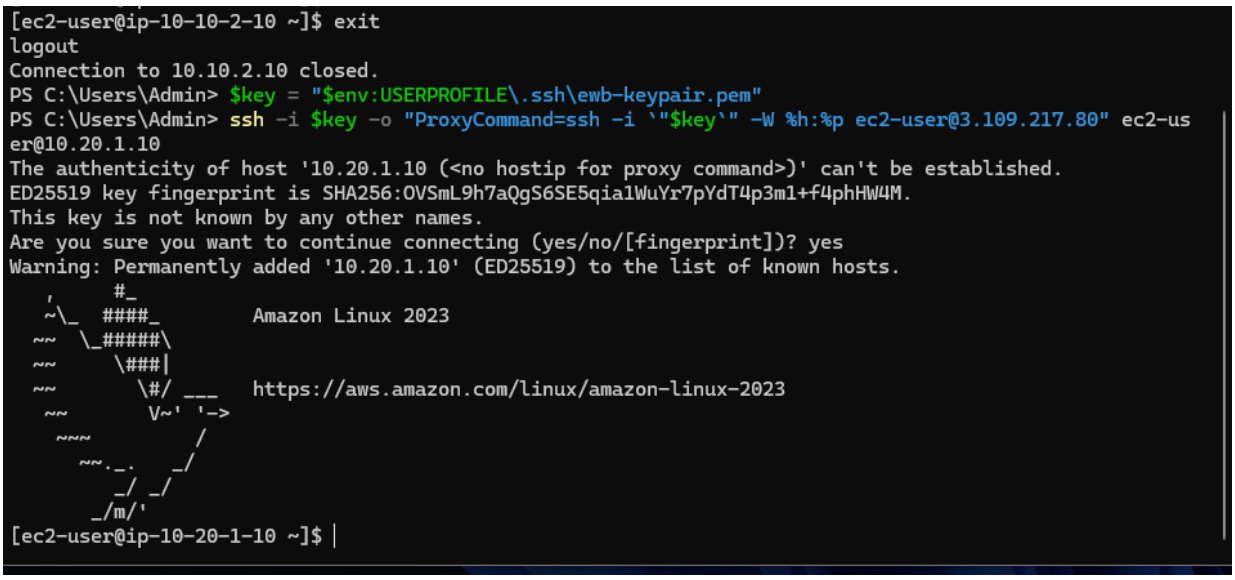


Figure 23. Successful SSH session on App-B through the bastion.

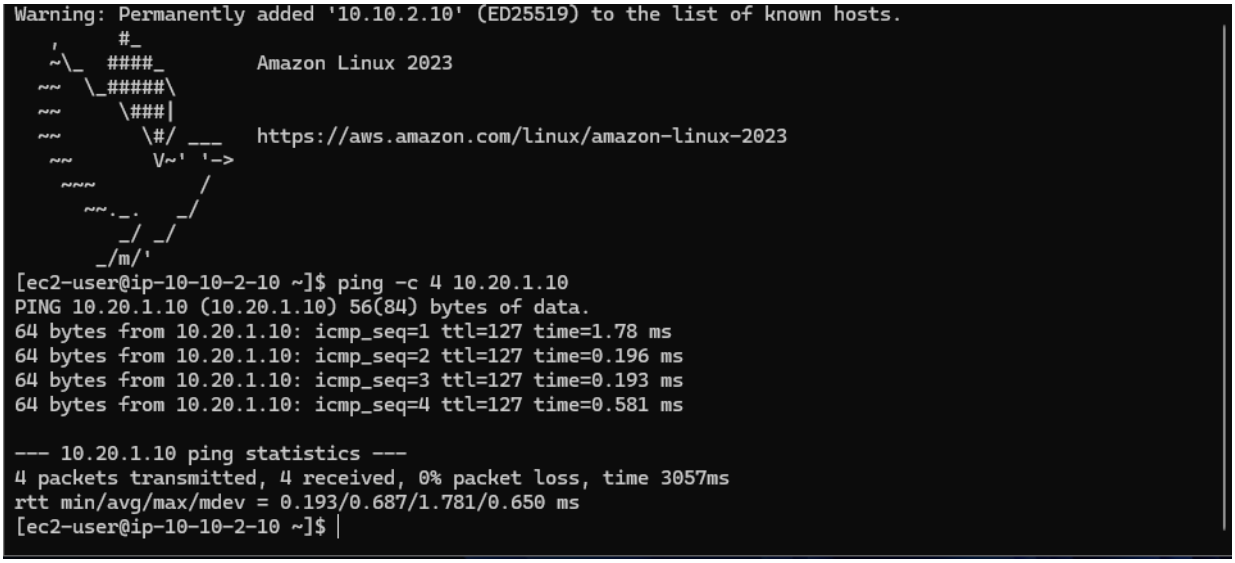


Figure 24. App-A ping to 10.20.1.10: 4 sent, 4 received, 0% packet loss.

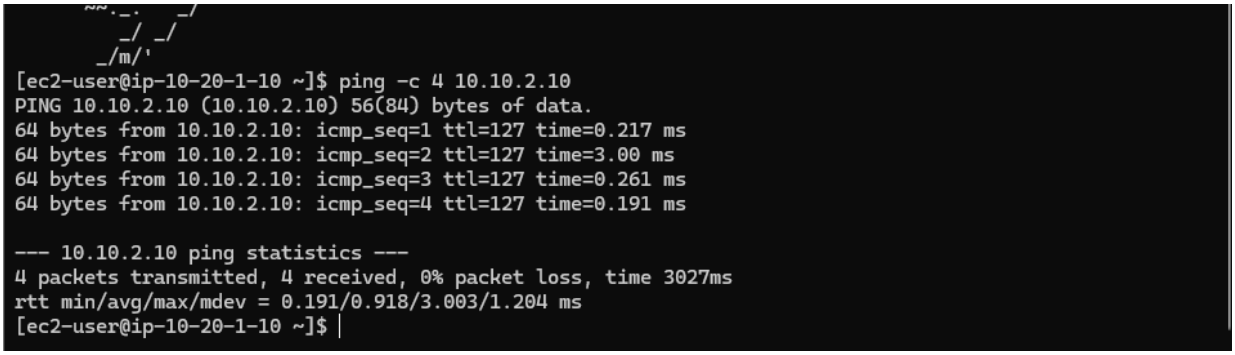


Figure 25. App-B ping to 10.10.2.10: 4 sent, 4 received, 0% packet loss.

Validation	Evidence / result
Bastion SSH	Successful Amazon Linux login shown
Bastion → App-A SSH	Successful App-A login shown
Bastion → App-B SSH	Successful App-B login shown
App-A → App-B ICMP	4/4 received; 0% loss
App-B → App-A ICMP	4/4 received; 0% loss

9. Troubleshooting and Lessons Learned

- A missing private-key path caused an SSH identity-file error. The downloaded key was located in a nested project folder rather than the Windows user .ssh directory.
- Windows OpenSSH rejected the key because its file ACLs were too permissive; restrictive permissions were applied before reconnecting.
- An attempted duplicate App-A inbound SSH rule was rejected; the correct TCP/22 rule from the bastion private IP already existed.
- An initial SSH connection toward App-A timed out while its instance status checks were still initializing. The instance readiness and attached security group were verified before retrying.
- A direct bastion-to-App-B ping failed earlier; this did not invalidate peering because the App-B ICMP rule was scoped to App-A, not the bastion.

10. Cleanup and Cost Awareness

After connectivity validation, all three EC2 instances were stopped. Stopping instances halts instance compute billing, but EBS storage, certain IP allocations, and other retained resources may continue to incur charges. Permanent resource deletion has not been evidenced in the supplied screenshots.

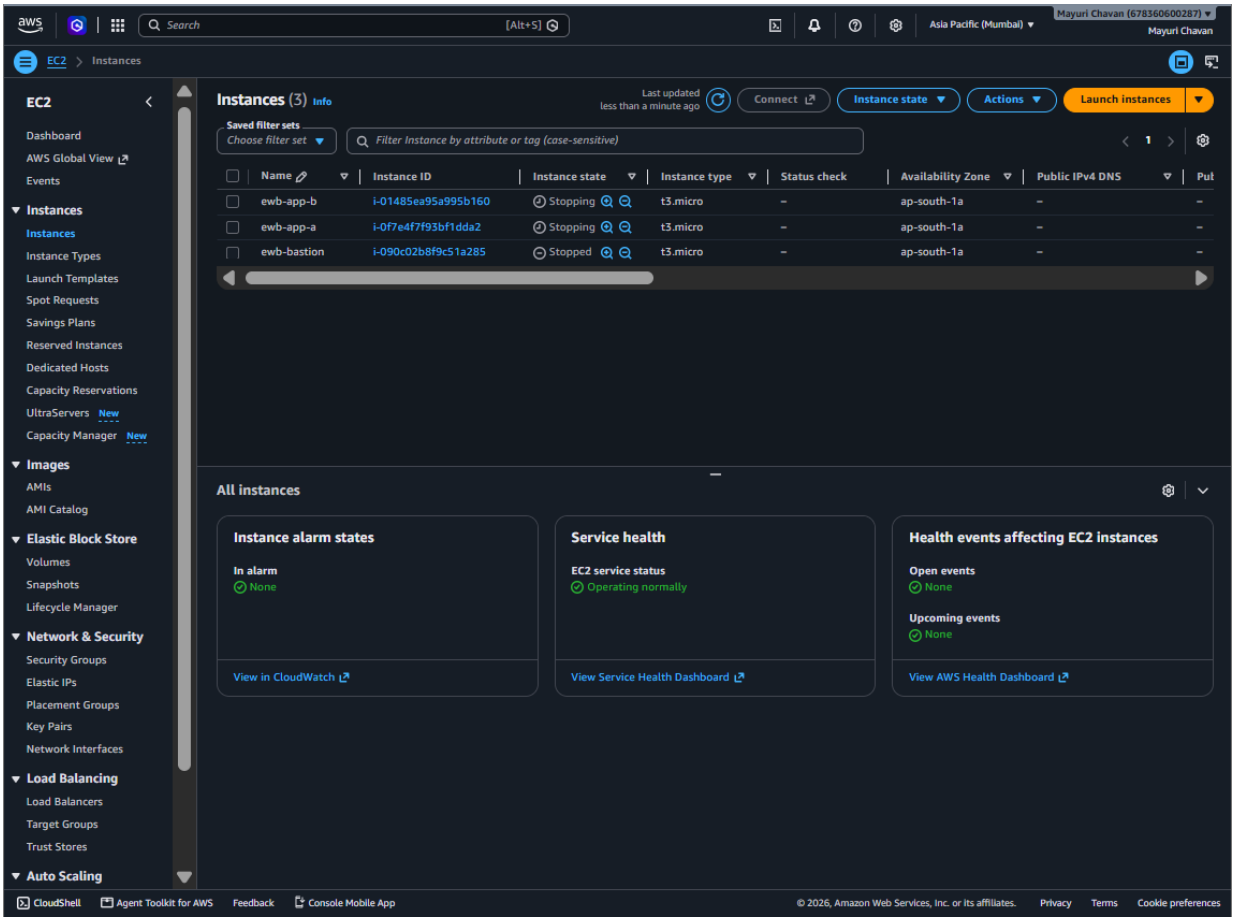


Figure 26. All three EC2 instances shown stopped after testing.

11. Results, Limitations, and Conclusion

The observed result is successful bidirectional private IPv4 reachability between application instances in distinct VPCs through an active VPC peering connection. Bastion-mediated SSH access avoided direct public exposure of the application instances. Route-table updates and narrowly scoped security groups were essential to the implementation.

Evidence limitations: The supplied screenshots do not document a controlled pre-peering failure test or an intentional ICMP-rule removal-and-recovery experiment. Those tests should not be claimed as completed. The report also does not establish permanent deletion of AWS resources.

Conclusion: The lab demonstrates practical AWS networking skills: CIDR planning, public/private subnet separation, bastion-based administration, VPC peering, route-table configuration, stateful security groups, and terminal-based connectivity validation. The result meets the core objective of secure communication between two otherwise isolated VPCs.

12. References

1. Amazon Web Services, Amazon VPC User Guide — VPC peering connections.
2. Amazon Web Services, Amazon EC2 User Guide — Connect to your Linux instance using SSH.
3. Amazon Web Services, Amazon VPC User Guide — Security groups and route tables.

4. EWB Cloud Labs — VPC Peering Student Project Workbook (project brief referenced in the lab discussion; original workbook not included in this upload).

Submission checklist: fill in institution/batch/mentor; verify screenshot readability; exclude all .pem files; decide whether additional workbook-specific test evidence is required; complete permanent AWS cleanup when no longer needed.